

7. Case Study—From NAND to XOR to Half-Adder

Gate-level error and hazard metric. At the gate level, we measure implementation quality with an error functional $\varepsilon(r)$ on a realization r . In this section, $\varepsilon(r)$ is interpreted not only as a logical mismatch rate against the intended truth table, but also as a timing-sensitive hazard indicator: transient glitches are counted when a circuit violates an admissible timing window and produces an unstable output during evaluation. This makes $\varepsilon(r)$ suitable for comparing alternative realizations that are logically equivalent but differ in depth, fanout, and susceptibility to hazards.

Deriving basic gates from NAND. The NAND gate is functionally complete, so the standard Boolean operations can be synthesized from it. Writing $x \uparrow y$ for NAND, one obtains

$$\neg a \equiv a \uparrow a, \quad a \wedge b \equiv (a \uparrow b) \uparrow (a \uparrow b), \quad a \vee b \equiv (a \uparrow a) \uparrow (b \uparrow b).$$

These identities exhibit the usual depth/size trade-off: a single primitive gate can generate the full Boolean basis, but the resulting implementations may require additional levels of composition. In a compositional setting, the relevant question is not merely whether a function is definable, but how the chosen realization affects structural complexity, delay, and robustness under the chosen $\varepsilon(r)$.

XOR from NAND and alternative forms. The exclusive-or operation is central to arithmetic circuits and can be expressed in the familiar sum-of-products form

$$a \oplus b \equiv (a \wedge \neg b) \vee (\neg a \wedge b).$$

This form makes the symmetry of XOR explicit: swapping a and b leaves the function unchanged. It also suggests a direct NAND synthesis by first constructing $\neg a$, $\neg b$, the two conjunctions, and then their disjunction. Alternative realizations are often preferable in practice. For example, one may use the identity

$$a \oplus b \equiv (a \vee b) \wedge \neg(a \wedge b),$$

or a purely NAND-based network with shared intermediate nodes. Different realizations are logically equivalent but may differ in gate count, depth, and hazard profile; the choice of implementation should therefore be evaluated against both functional correctness and the timing-window criterion encoded by $\varepsilon(r)$.

Half-adder specification and implementation. A half-adder takes two input bits $a, b \in \{0, 1\}$ and produces a sum bit s and a carry bit c . Its specification is

$$s = a \oplus b, \quad c = a \wedge b.$$

Thus the half-adder decomposes addition into a parity component and a carry component. Using the NAND-derived basis above, one can implement the half-adder entirely from NAND gates by reusing subexpressions for $\neg a$, $\neg b$, and $a \wedge b$, then assembling $a \oplus b$ from those intermediates. The resulting circuit is a concrete witness that a small universal primitive suffices to realize a standard arithmetic block.

Invariance under input swap. The half-adder specification is invariant under exchanging the two inputs:

$$(a, b) \mapsto (b, a).$$

Indeed, both $a \oplus b$ and $a \wedge b$ are commutative, so the output pair (s, c) is unchanged by swapping the inputs. This symmetry is not merely algebraic decoration; it is a useful correctness property for testing and for reasoning about implementation equivalence. Any candidate realization should preserve this invariance, and a failure to do so indicates either a wiring error or an unintended asymmetry introduced by the chosen decomposition.

Artifact: Verilog and testbench. The case study is intended to be accompanied by an executable artifact consisting of a Verilog implementation of the NAND-derived gates and half-adder, together with a testbench that enumerates the four input combinations $(a, b) \in \{0, 1\}^2$. The testbench should verify the truth table for s and c , and it should also check the swap invariance by comparing the outputs for (a, b) and (b, a) . In the presence of injected faults or timing perturbations, the same harness can be used to estimate $\varepsilon(r)$ empirically by counting mismatches and hazard events over the relevant input and timing scenarios.

Build-log perspective. From the standpoint of the book’s claims-as-builds protocol, this section is a small but representative build log. The specification begins with a Boolean relation, is refined into a NAND-only realization, and is then validated by executable tests. The important outputs are not only the final truth table, but also the intermediate identities, the structural choices that affect depth and sharing, and the invariance checks that expose accidental asymmetry. In this sense, the half-adder serves as a minimal benchmark for the broader methodology developed in the surrounding chapters.

Summary. Starting from NAND, one can derive negation, conjunction, disjunction, XOR, and finally a half-adder. The resulting chain of constructions illustrates the central theme of the book: closure under composition yields expressive power, while the choice of realization controls structural cost and robustness. The half-adder serves as a compact benchmark for both algebraic correctness and artifact-level testing under the error metric $\varepsilon(r)$.